

```
$ LD_LIBRARY_PATH=/usr/local/lib
$ export LD_LIBRARY_PATH
$ gcc -Wall first_gsl.c -lgsl -lgslblas -lm -o first_gsl
$ ./first_gsl
```

GSL has also created its own error constants and code to report errors in routines. Some of these are listed in `/usr/local/include/gsl/gsl_errno.h`, which you can read for a clearer understanding. You will see that there are some standard error codes used by some GSL functions, while we are given some freedom to use the GSL error functions to suit our needs. The standard error codes are interpreted in the respective algorithms where they are needed. For example, `GSL_ENOTSQR = 20`, is the error constant for 'matrix not square'.

You could also define your own error constants if none of the existing ones suit your error condition. There are a few functions available in these header files that you can use, or alter, to suit your needs. To demonstrate this, I have designed a simple program with two basic functions:

- Check if the number is positive or negative;
- If the number is negative, give an error; else, print the value of the number.

```
#include <stdio.h>
#include <math.h>
#include <gsl/gsl_errno.h> /*1. GSL header file includes error function
definition*/

int main (int argc, char *argv[])
{
    double x = -5.0;
    /*void gsl_error (const char * reason, const char * file, int line,
        int gsl_errno); This is the error function*/
    char reason[] = "Number is negative"; /*Error message*/
    char file[] = "error_gsl.c"; /*Error file name or argv[0] for
generalised case*/
    int line = 1; /*Line number*/
    int gsl_errno = -1;
    if(x>0)
    {
        printf("x is positive: %f\n",x);
    }
    else
    {
        x=fabs(x);
        gsl_error (reason, file, line, gsl_errno);
    }
    return 0;
}
```

Output (x is negative):

```
$ ./error_gsl
gsl: error_gsl.c:1: ERROR: Number is negative
```

Default GSL error handler invoked.
Aborted

Most of the program is self-explanatory. To make the program portable, you can include the definition of the function `gsl_error` in the same file, or make a separate header file of your own. The 'error.c' GSL file contains the `gsl_error` function definition (shown below). You can also change the function definition if you want.

```
void gsl_error (const char * reason, const char * file, int line, int gsl_errno)
{
    if (gsl_error_handler)
    {
        (*gsl_error_handler) (reason, file, line, gsl_errno);
        return ;
    }

    gsl_stream_printf ("ERROR", file, line, reason);
    fflush (stdout);
    fprintf (stderr, "Default GSL error handler invoked.\n");
    fflush (stderr);
    abort ();
}
```

The parameters for the function are: the reason for aborting, the file name in which the error occurred, the line number at which it occurred, and the error number (one of the pre-defined GSL error constants).



Tip: If you don't want the GSL error handler to abort your process, call the `gsl_set_error_handler_off()` function in your program, and add your own error-handling code.

I'd like to stress that you should try out both these programs yourself, and explore GSL on your own. If you want some more sample programs, look in the `/doc/examples` subfolder of the folder where you extracted the GSL archive.

In my next article, I will show you some of the great tools that are available with GSL! I personally recommend that you, as a FOSS user, should actively participate in the development of FOSS, including GSL!

It would be a great pleasure to help you with your doubts concerning GSL, if you do have any. My email address is below.

See you next month!  **END**

By: Amol Takkalki

The author is a FOSS enthusiast, whose interests include data mining, neural networks, process automation and high-performance/parallel computing. His hobbies are programming, photography, Linux and music. You can contact him at takkalki.amol@gmail.com. He strongly believes that, "Open source rocks!"